

Scalability & Reliability Design - Detailed

1. Design Goals and Numbers

Item	Value	Source
Normal traffic	~10,000 req/s	Brief
Flash-sale envelope	up to 500,000 req/s at the edge (50×)	Brief
Contending buyers	10,000 within ~1-3 s	Brief
Stock	100 units (Product X)	Brief
Payment success / failure	95% / 5%	Brief
Duplicate requests	2% (~200)	Brief
Order Service outage	30 s	Brief
Strict guarantees	no oversell, no duplicate reservation/payment/order, no lost paid order	Requirements

Core idea - a funnel: each layer removes load so that the single consistency point (the inventory row) sees only a handful of writes.

PART A - SCALABILITY

2. Request Funnel with Capacity Arithmetic (50× peak)

Stage	Input rate	Mechanism	Output rate	Disposition of the rest
1. Edge (CDN/WAF/bot)	500,000 req/s	Cache static + sale-state pages (TTL 1-2 s), bot scoring, per-IP limits	~25,000 req/s dynamic	~95% served from cache, ~2% blocked as bots
2. LB + API Gateway	25,000 req/s	JWT check, per-user limit (5 req/s), schema validation, Idempotency-Key required	~15,000 req/s valid	401/400/429 (rejected)
3. Waiting room	15,000 req/s	Token bucket admits ≤ 2,000 buy attempts/s; randomised queue position	2,000 req/s	202 + queue position (queued) or shed
4. Redis stock gate	2,000 req/s	Lua atomic decrement; first 100 pass	≤ ~100 + retries	~1,900/s 409 OUT_OF_STOCK in microseconds
5. Inventory DB	~100 txns total	Atomic conditional UPDATE + reservation INSERT	100 reservations	OUT_OF_STOCK if 0 rows
6. Payment	≤ 100 intents	Paced to gateway rate limit via queue	~95 success	~5 failures release stock
7. Order & downstream	~95 events	Kafka, async	95 orders	Buffered during outages

Why this works: the cost of rejecting a request is O(1) in memory (Redis/edge), whereas serving it in the DB would be a lock wait. **DB write load is independent of traffic volume** (it depends only on stock), so a 50× increase affects the edge and gateway, not the consistency layer.

3. Where Requests Are Throttled, Queued, Rejected

Control	Layer	Behaviour
Throttle	WAF, Gateway	Per-IP, per-user, per-endpoint token buckets; 429 with Retry-After
Queue	Waiting room	Admission tokens, queue position, long-poll/SSE updates; randomised order after the sale opens to avoid latency-based unfairness
Queue	Kafka	Post-payment events (order, shipment, notification) buffered
Reject	WAF	Bots, malformed requests
Reject	Redis gate	OUT_OF_STOCK
Reject	DB	0 rows updated → OUT_OF_STOCK
Shed	Gateway	Non-critical APIs first when latency SLO breached

Exact point of inventory contention control: the inventory row, via UPDATE ... WHERE available_quantity >= :q inside one local transaction. Everything before is a filter that is allowed to be wrong in the safe direction.

4. Horizontal Scaling and State Management

Layer	Scaling method	Scaling signal	State location
CDN	Provider-managed	-	Cached content
Gateway	HPA, pre-scaled	CPU, RPS, p99 latency	Stateless (tokens validated locally, rate-limit counters in Redis)
Admission	HPA	queue depth	Redis (queue, tokens)
Product/Sale, Cart	HPA	RPS	Redis cache; catalogue read replicas
Inventory	HPA (stateless pods)	RPS, DB pool wait	PostgreSQL primary

Checkout, Payment, Order	HPA / KEDA	CPU, Kafka lag	Own DBs + Kafka
Redis	Cluster shards, replicas	ops/s, memory	Ephemeral; rebuilt from DB
PostgreSQL	Vertical for hot inventory primary; partitioning for orders/payments; read replicas	CPU, connections, replication lag	Durable
Kafka	Partitions ≥ consumers; add brokers	Consumer lag, bytes in	Durable log

Statelessness rules: no sticky sessions; no in-memory business state; all idempotency/dedupe state in Redis or DB; configuration externalised; graceful shutdown drains in-flight requests.

5. Capacity Sizing (illustrative, to be validated by a load test)

Component	Assumption	Result
Gateway pod	~2,500 req/s per pod	25,000 req/s → 10 pods, ×3 headroom = 30 pods across 3 AZs
Admission service	2,000 admits/s, light CPU	6 pods
Inventory pods	≤ 2,000 req/s at the gate	6 pods; DB pool ≤ 20 connections per pod via PgBouncer
Redis	100k+ ops/s per shard	3 shards × 1 replica
Postgres inventory	≈ 100 write txns for the sale	trivial; sized for steady-state + safety
Kafka	~200 events/s peak	3 brokers, 12 partitions per topic

These are starting values, not measured results; the pre-sale load test (Locust/JMeter) validates them.

6. Bottleneck Analysis

#	Bottleneck	Why	Mitigation	Residual risk
1	Hot inventory row (most likely)	All writers touch one row	Admission control + Redis gate (only ~100 reach DB); single-statement critical section; per-customer limit	Multi-thousand-unit SKUs: use stock bucketing (below)
2	Payment gateway rate/latency	External	Queue/pacing, circuit breaker, secondary provider, TTL grace	Gateway-wide outage
3	API gateway/LB connection surge	Refresh storms	Pre-scale, connection limits, CDN absorb, shed	Misconfigured limits
4	Redis hot key	One key for stock	One key handles >100k ops/s; replicas for reads; fallback to bucketing	Redis failure → gate bypass (safe)
5	Kafka consumer lag	Post-payment spike	KEDA autoscale on lag, partitions sized for peak	Poison messages → DLQ
6	DB connections	Many pods	PgBouncer transaction pooling, per-service pools	Pool exhaustion → backpressure
7	Polling thundering herd	Users refresh/poll	Cached sale state, SSE/WebSocket push, jittered <code>retryAfter</code>	–

6.1 Stock bucketing (escalation path for very large SKUs)

Split `total=N` into `B` bucket rows (`inventory_bucket(product_id, bucket_id, available)`); route a request by `hash(customer_id) mod B`; if the bucket is empty, try the next bucket(s); the sum of buckets always equals total. Trade-off: "sold out" detection needs scanning all buckets (done by the reconciler), and a few units may be stranded in a nearly-empty bucket until the scan.

7. Database Scaling Strategy

Concern	Strategy
Inventory writes	Single primary + synchronous standby (RPO 0); logic keeps writes to the minimum
Reads (catalogue, order history)	Read replicas, Redis cache
Orders/payments growth	Time-based partitioning (monthly), archive after N months
Indexes	Partial index for expiry sweep (<code>state IN (RESERVED, PAYMENT_PENDING)</code>), unique indexes for idempotency
Connection pooling	PgBouncer; bounded pool per service
Hot-row contention	Short transactions (< 10 ms), no external calls inside a transaction
Future horizontal scale	Shard by <code>product_id / sale_id</code> (inventory per sale is naturally partitionable)

8. Caching Strategy

Data	Where	TTL / invalidation	Used for decisions?
Static assets, sale page	CDN	60 s, stale-while-revalidate	No
Stock banner ("Few left"/"Sold out")	CDN + Redis read model	1–2 s; updated on <code>StockExhausted</code> , <code>ReservationReleased</code>	No
Stock gate counter	Redis	Initialised from DB at sale open; adjusted on events; drift check every 10 s	Fast rejection only
Cart	Redis	30 min TTL, snapshot at checkout	No
Idempotency fast check	Redis (24 h)	DB unique constraint is the authority	Fast replay only
Product detail	Redis + CDN	5 min; purge on update	No

Cache-failure rule: **no cache outage may cause oversell**; the worst effect is more load on the DB, controlled by admission rate.

9. What Changes at 50× Traffic (10k → 500k req/s)

Area	Change	Reason
Edge	Multi-CDN or high-capacity CDN plan, pre-rendered "Sold out" page, aggressive TTLs	>95% of requests must never reach origin
Gateway	10-50× pods pre-warmed; HTTP/2; keep-alive; per-region autoscale limits raised	Absorb connection surge
Admission	Mandatory; sharded by <code>sale_id</code> ; randomised queue	Bounded concurrency; fairness
Rate limits	Stricter per-user/IP; proof-of-work/CAPTCHA at sale start	Bot/abuse control
Redis	More shards, local gateway cache for sale state; stock key on dedicated shard with replicas	Hot key isolation
DB	No change in write load ; larger standby, PgBouncer limits	Funnel keeps writes ≈ stock
Kafka	More partitions/consumers; consumer autoscale	Post-payment burst
Payment	Pacing queue sized to gateway limits; second provider warmed	External rate limit
Load shedding	Drop recommendations/reviews/analytics first	Protect buy path
Observability	Sampling lower for browse; 100% for buy path	Cost control
Operations	Game-day rehearsal, change freeze, war-room dashboards	Human readiness
Unchanged	Correctness mechanism (atomic UPDATE), idempotency, saga	Independent of scale

10. Trade-offs Accepted

We gain	We give up
Strong inventory correctness	Some users wait in queue; fairness approximate
Resilient async pipeline	Seconds of eventual consistency between payment and order
Fast rejection through cache	Displayed stock may be briefly stale
Single authority for stock	Hot-row limit for huge SKUs (bucketing escape hatch)
Defence in depth	Operational complexity and cost

PART B - RELIABILITY AND FAILURE HANDLING

11. Required Test Case Walk-Through (Stock 100, Users 10,000, 95% / 5%, 2% duplicates, Order Service down 30 s)

Step	Event	Counts / result
1	10,000 buy requests arrive	~200 are duplicates (same key)
2	Duplicates	Return the original reservation or result; 0 extra stock
3	Gate + DB	100 reservations created (<code>RESERVED</code>); ~9,700-9,900 <code>OUT_OF_STOCK</code>
4	Payments initiated	100
5	95 succeed	<code>PaymentSucceeded</code> events; reservations → <code>CONFIRMED</code>
6	5 fail	<code>PaymentFailed</code> → <code>RELEASED</code> ; stock +5; Redis tokens returned
7	Waitlist	First 5 queued users get a reservation window; sale can end at exactly 100 sold
8	Order Service down for 30 s	Events retained in Kafka; reservations remain <code>CONFIRMED</code> (exempt from expiry); users see "payment received, confirming order"
9	Recovery at T+30 s	Redelivery → 95 orders created idempotently → <code>OrderConfirmed</code> → <code>SOLD</code>
10	Final state	<code>sold = 100</code> at most; <code>available + reserved + sold = total</code> ; no payment without order

12. Mandatory Demonstration Traces

12.1 Successful purchase

Request (key K1) → gateway → admission → duplicate check (none) → Redis token → DB: `UPDATE inventory + INSERT reservation (RESERVED)` + outbox → `201 RESERVED` → checkout (price, payment `INITIATED-PENDING`) → customer pays → webhook verified, deduped → payment `SUCCEEDED` + outbox → Inventory: `CONFIRMED` → Order created (unique on `reservation_id`) → `OrderConfirmed` → Inventory `SOLD`, Shipment, Notification.

12.2 Failed payment and release

Webhook failed → payment `FAILED` + outbox `PaymentFailed` → Inventory guarded update `RESERVED/PAYMENT_PENDING` → `RELEASED` (reason `PAYMENT_FAILED`) → `available+1, reserved-1` in the same transaction → Redis `INCRBY` → `ReservationReleased` → waitlist and customer notification.

12.3 Duplicate Buy request

Second request with the same `Idempotency-Key`: lookup finds the stored reservation → `200 {replayed:true}`; concurrent duplicates: one `INSERT` wins, the other violates `UNIQUE(customer_id, idempotency_key)`, its transaction rolls back (stock decrement is undone atomically), re-read returns the winner's reservation.

12.4 Payment success followed by Order Service failure

Payment row SUCCEEDED and outbox row commit atomically → Kafka retains PaymentSucceeded → Order consumer unavailable, offset not advanced → recovery at T+30 s redelivers → idempotent create. Safety nets: OrderRecoveryJob (payments SUCCEEDED > 2 min without order), DLQ after 5 failed retries, and compensation (refund + release) only when an order can never be created.

12.5 Inventory reaches zero

Redis counter = 0 → StockExhausted event → CDN/cache flips to "Sold out"; Admission stops admitting; later requests receive 409 OUT_OF_STOCK without DB access. If a reservation expires or a payment fails, available > 0 again → state "Back in stock" for queued/waitlisted users only.

12.6 Reservation expiry

Sweeper every 5 s: SELECT ... WHERE state IN (RESERVED, PAYMENT_PENDING) AND expires_at <= now() FOR UPDATE SKIP LOCKED LIMIT 500 → guarded update to RELEASED (TIMEOUT) → stock returned → events. A payment that arrives after release: if the unit is still free, atomically re-reserve and confirm; otherwise auto-refund (ADR-013).

13. Failure Scenarios and Recovery Matrix

#	Failure	Detection	Behaviour during failure	Recovery	RTO / RPO
1	Service pod crash	Liveness/readiness	LB removes pod; retries hit others	K8s restarts; stateless	seconds / 0
2	DB primary failure	Patroni health, errors	Writes fail ≤ ~30 s: gateway 503 retryable ; reads from cache/replicas; payments in flight stay PENDING	Sync standby promoted; services reconnect; reconcile PENDING payments; re-sync Redis from DB	< 15 min / 0
3	DB standby failure	Replication alert	Primary continues per policy	Rebuild standby	-
4	Payment gateway down	Breaker OPEN, error rate	Fail fast; payments PENDING_RETRY ; TTL grace once; message "payment service busy"; failover provider if configured	Half-open probes; reconciler retries with same merchant_ref	gateway-dependent
5	Gateway timeout (single call)	Timeout	Status unknown → TIMED_OUT , never failed	Query by merchant_ref , finalise	minutes
6	Redis down	Health/timeouts	Gate bypassed; admission rate lowered; DB still correct	Rebuild counter from DB available_quantity	seconds
7	Redis counter drift	Drift check (10 s)	Possible false rejects or extra DB attempts (never oversell)	Overwrite from DB	-
8	Kafka unavailable	Producer errors	Outbox rows remain unpublished (state safe)	Relay resumes, no loss	- / 0
9	Order Service down	Lag, health	Events retained; reservations stay CONFIRMED	Redelivery + recovery job	30 s case
10	Poison message	Retry exhaustion	Moved to DLQ after 3-5 tries; partition continues	Fix, replay	-
11	Notification/courier down	Breaker	Retry with backoff/fallback	Non-critical	-
12	Duplicate/late webhook	gateway_event dedupe	Ignored	-	-
13	Region failure	Health checks	Fail over to warm DR region	Promote DR DB, scale pods	< 30 min / small (async only)
14	Orchestrator crash mid-saga	Saga deadline	Saga state persisted	Resume or compensate	-

14. Retry, Timeout and Circuit-Breaker Policy

Dependency	Timeout	Retry	Backoff	Breaker
Inventory (from Checkout)	500 ms	1 (idempotent key)	none	yes
Payment gateway	connect 1 s / read 5 s	3, same merchant_ref	exponential + jitter (200 ms base)	open at 50% failures / 20 calls or 10 consecutive timeouts; half-open after 20 s with 3 probes
Courier / Notification	3 s	5	exponential	yes
Kafka produce (relay)	5 s	unlimited	capped exponential	n/a
Consumers	-	3 then DLQ	1 s, 5 s, 30 s	-

Retry rules: retry only transient errors (network, 5xx); never retry 4xx declines; never retry a charge with a new reference; add jitter to avoid synchronised retries.

15. Dead Letters, Compensation and Reconciliation

Mechanism	Detail
DLQ	One per consumer group; alert when depth > 0; replay tool re-injects after fix (idempotency makes replay safe)
Compensation	Payment ok but order impossible → refund + release; payment failed → release; order cancelled → refund + restock
Reconciliation (every 1 min / hourly full)	(1) expired reservations not released; (2) PENDING/TIMED_OUT payments vs gateway; (3) SUCCEEDED payments without order; (4) invariant available + reserved + sold = total and sold = count(SOLD) ; (5) Redis counter vs DB; findings emit ReconciliationFinding alerts, safe cases auto-repair

16. Graceful Degradation Order (shed first → last)

recommendations/reviews → cart enrichment → analytics → delay notifications → delay shipment creation → **never shed**: reserve, pay, order confirmation.

17. Disaster Recovery

Item	Design
Backups	PITR for databases (continuous WAL archive), daily snapshots, Kafka mirrored
DR region	Warm standby: async DB replica, MirrorMaker, scaled-down pods
Failover	Runbook-driven promote; DNS/traffic switch; sale paused during failover
Testing	Quarterly failover drill; game day before every major sale

18. Runbooks (summary)

Situation	First actions
Inventory invariant alert	Pause sale (admin API), run reconciliation, compare <code>inventory</code> vs reservations, repair, resume
Payment failure spike	Check breaker/gateway status, switch provider, extend TTL, inform customers
Queue/lag alert	Scale consumers, inspect DLQ, check downstream health
DB failover	Confirm promotion, restart connection pools, run payment reconciliation

19. Jury Q&A

Q: What is the bottleneck and how does it scale? The hot inventory row; the funnel keeps its load at $\approx$ stock size; bucketing handles large stock. **Q: What if Redis lies?** The DB conditional UPDATE and CHECK constraints still enforce correctness; Redis is rebuilt from the DB. **Q: What changes at 50x?** Edge, gateway, admission and Kafka scale out; DB write load is unchanged. **Q: How do you prove no oversell?** The invariant `available + reserved + sold = total` as a CHECK constraint, plus the reconciliation job and an alert on any violation.